

Tugas 02_audio_noise_statis

Nama: Muhammad Piela Nugraha

NIM: 123140200

Tautan Repository GitHub: <https://github.com/EL-graha26/stm-#f25-40305-123140200>

Dalam tugas ini saya diminta merekam suara (voice recorder) saya sendiri sedang membaca berita dari Google, dengan kondisi rekaman mengandung noise, yaitu suara kipas angin (derau statis / static noise) di latar belakang.

Akuisi Audio

Saya Memasukan hasil rekaman saya membaca berita ke komputer dengan parameter sr=None (sample rate) ini agar komputer membaca audio sesuai dengan kualitas aslinya saat di rekam tanpa mengubah apapun tahap persiapann library dan audio

```
In [1]: #mengimport semua bahan yang dibutuhkan
import librosa          # Library untuk olah audio (baca, tulis, ekstraksi fitur, dll)
import librosa.display   # sub-library dari librosa untuk menampilkan grafik spektrum audio
import numpy as np       # library untuk olah array/matriks (bisa buat operasi matematika, statistik, dll)
from pathlib import Path  # library untuk mengelola path file dengan mudah
import matplotlib.pyplot as plt #library untuk menampilkan grafik (plot) hasil olah audio
import scipy.io.wavfile as wavfile # library untuk membaca dan menulis file audio format .wav

print("library ready")
```

library ready

```
In [13]: #menimpa file audio yang akan diolah
audio = "audio_original.wav"
audio_path = Path(audio)

# membaca file audio
y, sr = librosa.load(str(audio_path), sr=None)
print(f"audio : {audio_path}")

audio : audio_original.wav
```

```
In [14]: #menganalisis informasi dasar file audio
durasi = len(y) / sr #jumlah titik data dibagi dengan laju sampel = perdetik

print(f"laju sampel asli (fs): {sr} Hz")
print(f"Durasi: {durasi:.2f} detik")
print(f"Total sampel titik data: {len(y)}")
print(f"Amplitudo Min: {np.min(y):.4f}, Max: {np.max(y):.4f}")

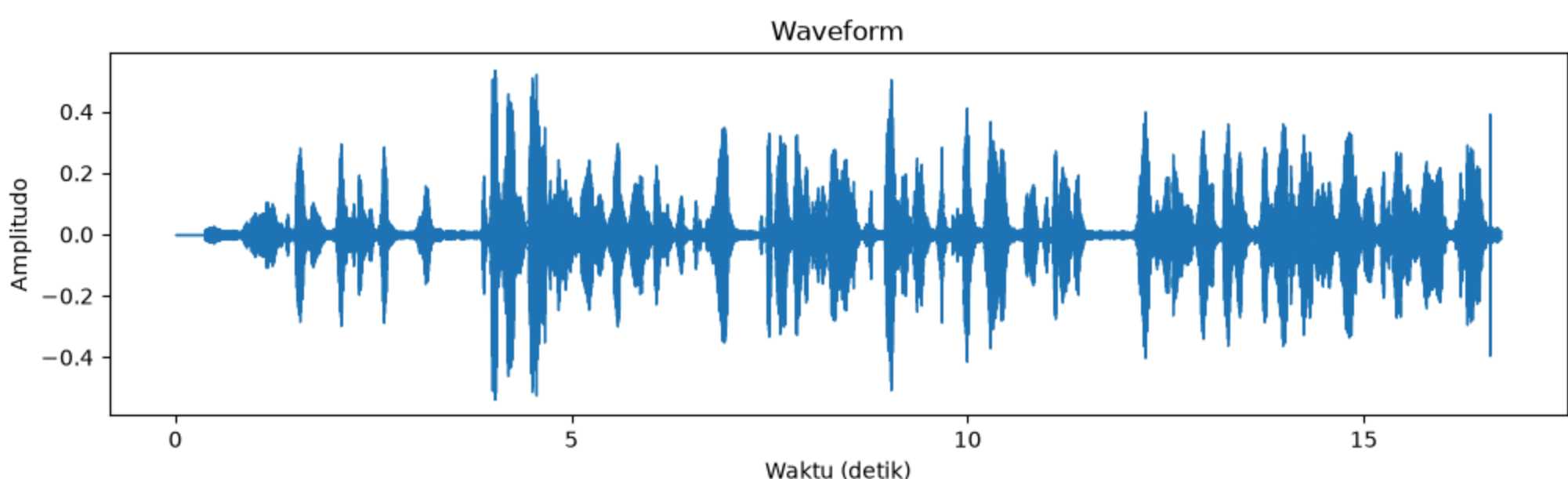
Laju sampel asli (fs): 48000 Hz
Durasi: 16.73 detik
Total sampel titik data: 802815
Amplitudo Min: -0.5116, Max: 0.5363
```

Menvisualisasi audio

Waveform

bentuk gelombang suara yang menunjukkan perubahan amplitudo terhadap waktu.

```
In [15]: plt.figure(figsize=(12, 3)) # membuat gambar dengan ukuran 12x3 inci
librosa.display.waveshow(y, sr=sr) #menampilkan grafik waveform dari sinyal audio terhadap waktu
plt.title("Waveform")
plt.xlabel("Waktu (detik)")
plt.ylabel("Amplitudo")
plt.show()
```



Spektrum FFT

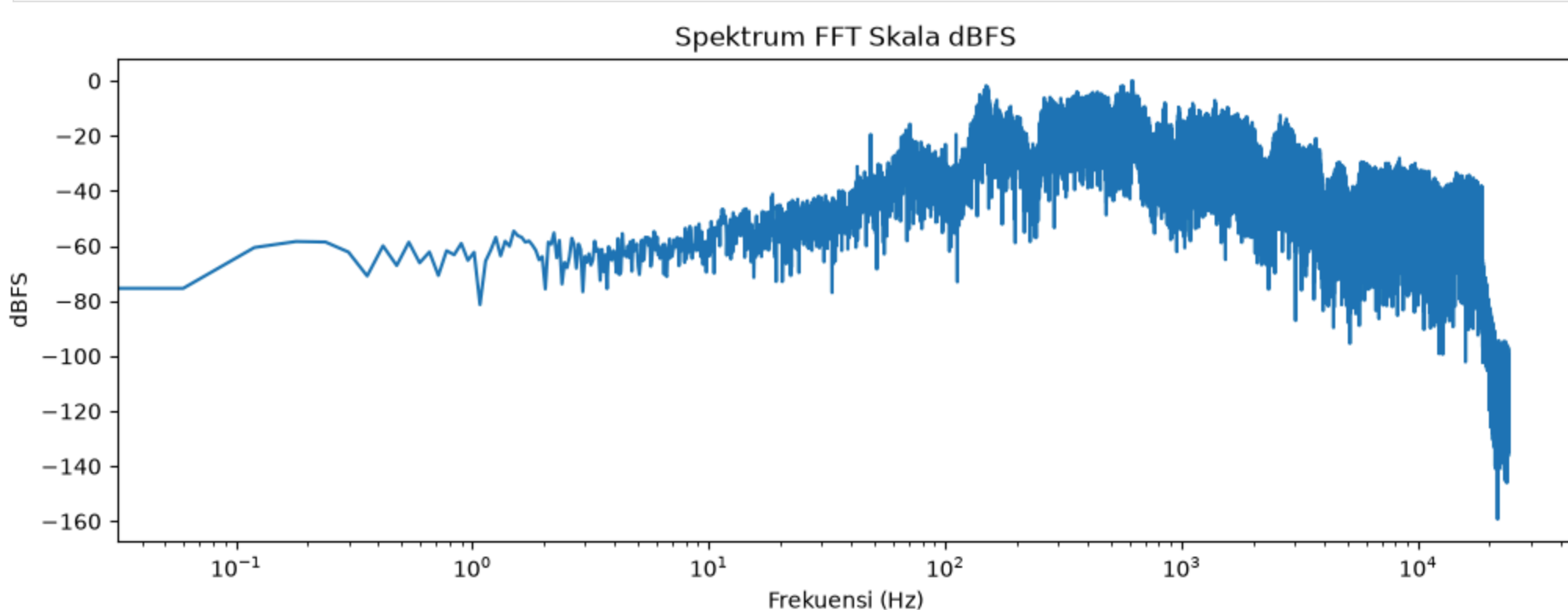
mengubah sinyal dari domain waktu ke domain frekuensi, sehingga frekuensi yang ada pada rekaman bisa dilihat.

```
In [17]: # FFT mengubah sinyal dari domain waktu ke domain frekuensi
Y_fft = np.fft.fft(y) # hasil FFT (berupa angka kompleks)
f_freqs = np.fft.fftfreq(len(y), 1/sr) # frekuensi yang disesuaikan dengan hasil FFT Hz
magnitude = np.abs(Y_fft) #berdasarkan nilai kekuatab signal (magnitude)

# Mengubah magnitude ke skala dBFS
magnitude_dbfs = 20 * np.log10(magnitude / np.max(magnitude) + 1e-9)

plt.figure(figsize=(12, 4))
plt.plot(f_freqs, magnitude_dbfs)
plt.title("Spektrum FFT Skala dBFS")
plt.xlabel("Frekuensi (Hz)")
plt.ylabel("dBFS")

# Menggunakan skala log agar frekuensi rendah lebih terlihat
plt.xscale('log')
plt.show()
```



Menemukan Frekuensi Dominan Noise

Mengambil bagian awal audio yang masih hening untuk mencari frekuensi noise yang paling dominan.

```
In [18]: durasi_sampel_hening = 0.5 # durasi sampel hening dalam detik
n_sampel_hening = int(durasi_sampel_hening * sr) # mengubah durasi menjadi jumlah sampel (titik data) berdasarkan laju sampel
segmen_hening = y[0:n_sampel_hening] # mengambil bagian awal saja

fft_hening = np.abs(np.fft.rfft(segmen_hening)) #menghitung FFT dari segmen hening dan mengambil magnitudenya
freqs_hening = np.fft.rfftfreq(len(segmen_hening), 1/sr) #menentukan frekuensi gening

idx_dominan = np.argmax(fft_hening)
freq_dominan = freqs_hening[idx_dominan]

print(f"Frekuensi paling dominan dari noise statis (bagian hening): {freq_dominan:.1f} Hz")
```

Frekuensi paling dominan dari noise statis (bagian hening): 70.0 Hz

Analisis Karakteristik Derau Statis

Dari hasil FFT pada 0,5 detik pertama saat kondisi hening, ditemukan frekuensi noise dominan sekitar 70 Hz.

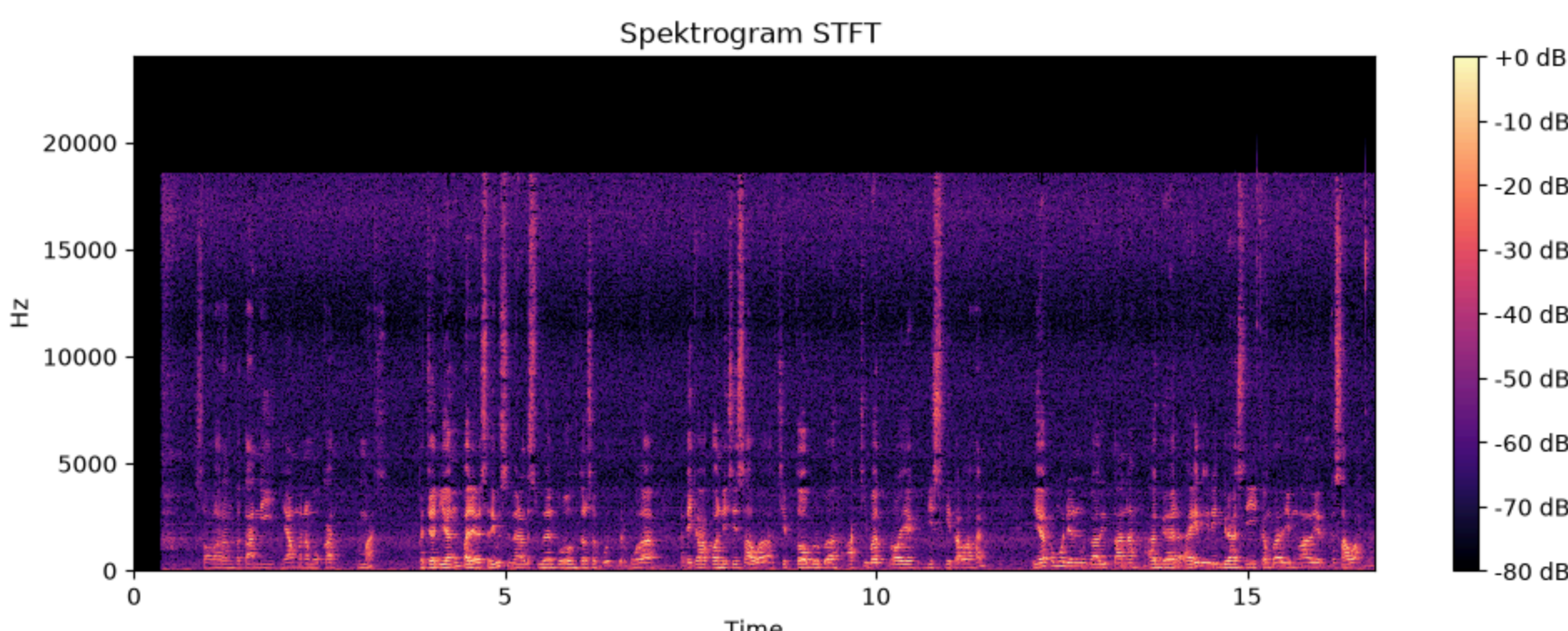
- Noise berada pada frekuensi rendah dan kemungkinan berasal dari kipas/motor listrik.
- Pada spektrum FFT, energi di sekitar frekuensi rendah terlihat lebih tinggi dan stabil, sehingga noise bukan *white noise*, tetapi lebih cenderung *colored noise*.
- Pada spektrogram dan Mel-spektrogram, noise terlihat sebagai lapisan energi yang terus muncul sepanjang rekaman, termasuk saat tidak ada suara.
- Berbeda dengan suara bicara yang muncul dan menghilang, noise ini bersifat kontinu.
- Noise seperti ini dapat dikurangi menggunakan high-pass filter atau noise reduction berdasarkan profil noise dari bagian hening.

Spektrogram STFT

melihat perubahan frekuensi suara dari waktu ke waktu.

```
In [19]: D = librosa.stft(y)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max) # mengubah hasil STFT ke skala dBFS (Decibels Full Scale)

plt.figure(figsize=(12, 4))
librosa.display.spectrogram(S_db, sr=sr, x_axis='time', y_axis='hz')
plt.colorbar(format='%+2.0f dB') # Menampilkan skala kekuatan sinyal
plt.title("Spektrogram STFT ")
plt.show()
```

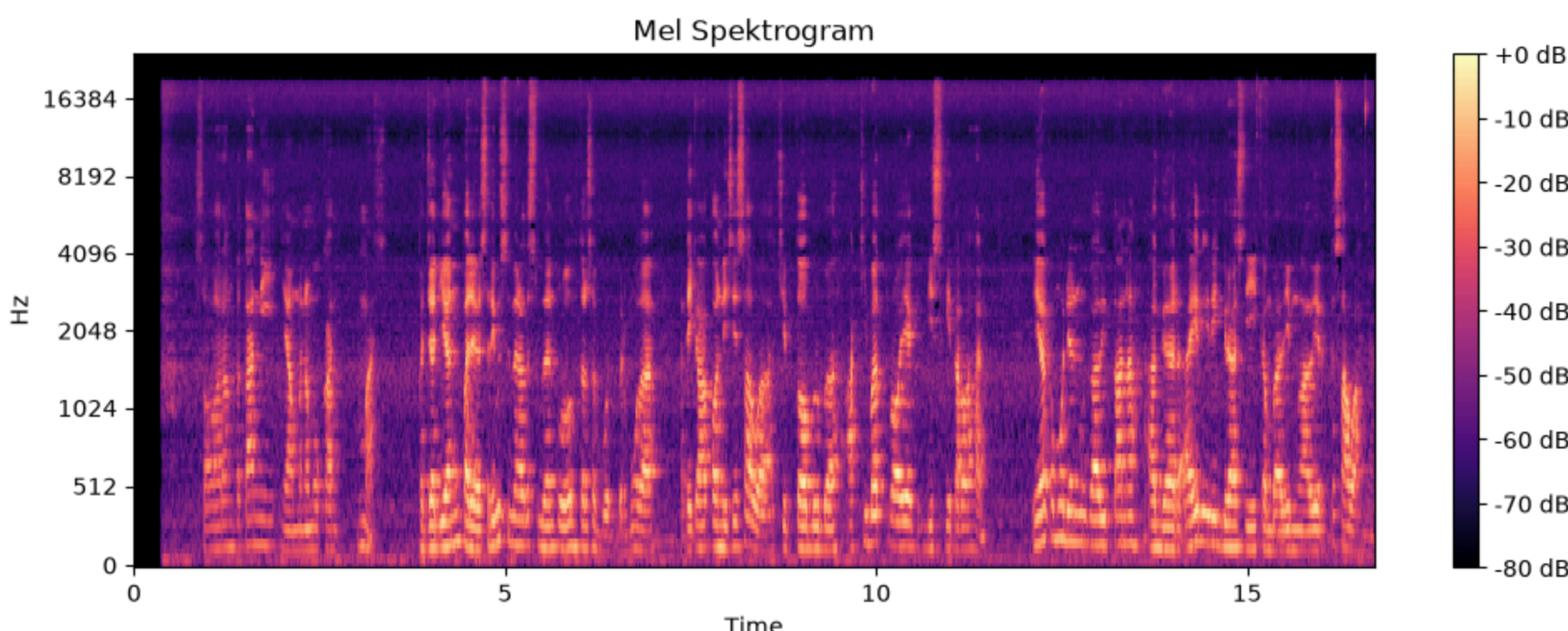


Mel Spektrogram

frekuensi suara dengan skala Mel yang disesuaikan dengan pendengaran manusia.

```
In [20]: M = librosa.feature.melspectrogram(y=y, sr=sr) # Membuat Mel-Spektrogram dari audio
M_db = librosa.power_to_db(M, ref=np.max) # Mengubah hasil ke skala desibel

plt.figure(figsize=(12, 4))
librosa.display.spectrogram(M_db, sr=sr, x_axis='time', y_axis='mel')
plt.colorbar(format='%+2.0f dB') # Menampilkan skala kekuatan sinyal
plt.title("Mel Spektrogram")
plt.show()
```



Resampling dan aliasing audio

dimana menurunkan kualitas audio dengan dua cara berbeda lalu membandingkan hasilnya

```
In [26]: # A. downsampling (naïve decimation)
# Menurunkan sr audio ke 8.000 Hz dan mengambil sebagian sampel tanpa filtering
target_sr = 8000 # Menentukan laju sampel target (misalnya 8000 Hz)
faktor_potong = sr // target_sr # Menentukan faktor pengurangan sampel
y_kasar = y[::faktor_potong] # Mengambil setiap sampel ke-n
print(f" A selesai Jumlah sampel: {len(y_kasar)}")

A selesai Jumlah sampel: 133803
```

```
In [27]: # B. resampling audio dengan filter
#Menurunkan sr menggunakan resampling dengan filter untuk mengurangi risiko aliasing.

y_bersih = librosa.resample(y, orig_sr=sr, target_sr=target_sr)
print(f" B selesai Jumlah sampel setelah resampling halus: {len(y_bersih)}")

B selesai Jumlah sampel setelah resampling halus: 133803
```

```
In [29]: #menyimpan file audio resampling
wavfile.write("audio_downsampled_naive.wav", target_sr, (y_kasar * 32767).astype(np.int16))
wavfile.write("audio_downsampled_clean.wav", target_sr, (y_bersih * 32767).astype(np.int16))

print("file a dan b telah di simpan ")

file a dan b telah di simpan
```

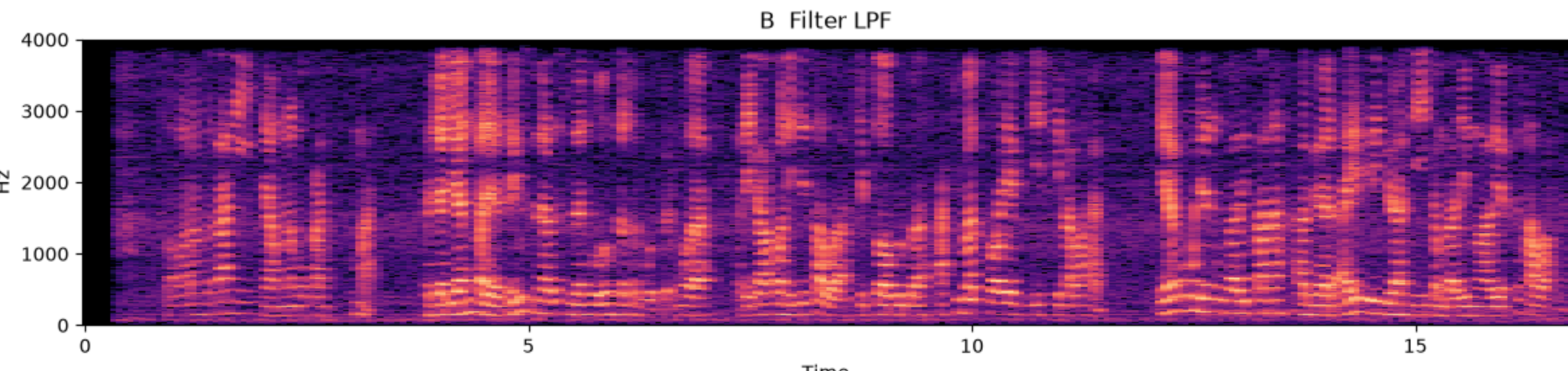
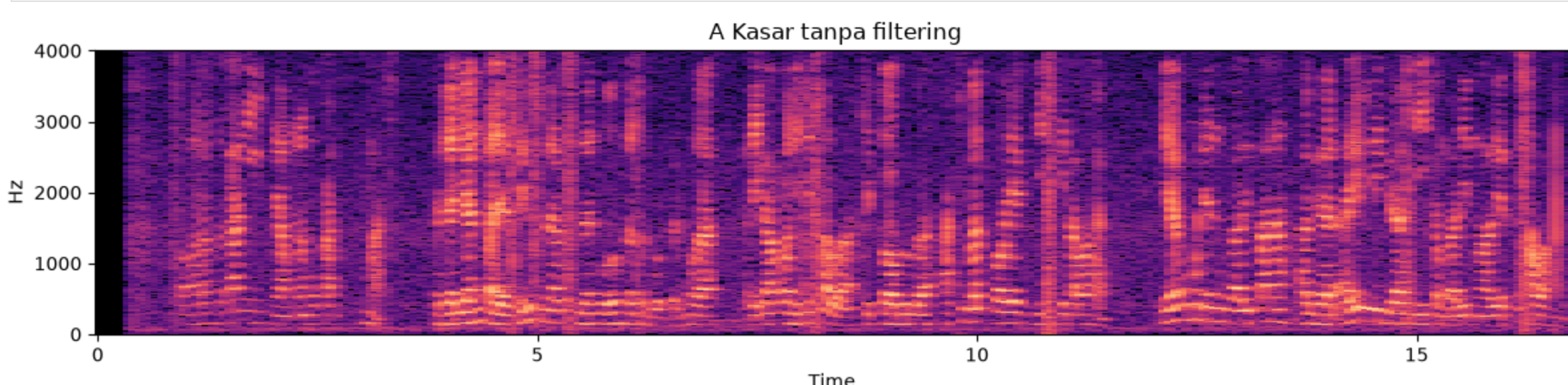
Spektrogram dengan filtering dan tanaoa filtering

```
In [31]: plt.figure(figsize=(12, 6))

plt.subplot(2, 1, 1)
D_kasar = librosa.stft(y_kasar)
librosa.display.spectrogram(librosa.amplitude_to_db(np.abs(D_kasar), ref=np.max), sr=target_sr, x_axis='time', y_axis='hz')
plt.title(" A Kasar tanpa filtering")

plt.subplot(2, 1, 2)
D_bersih = librosa.stft(y_bersih)
librosa.display.spectrogram(librosa.amplitude_to_db(np.abs(D_bersih), ref=np.max), sr=target_sr, x_axis='time', y_axis='hz')
plt.title(" B Filter LPF ")

plt.tight_layout()
plt.show()
```



Analisis Temuan Aliasing pada Downsampling

Sinyal asli memiliki sample rate 48.000 Hz dan diturunkan menjadi 8.000 Hz, sehingga frekuensi Nyquist baru adalah 4.000 Hz. Frekuensi di atas 4.000 Hz berisiko mengalami aliasing jika tidak difilter terlebih dahulu.

Perbandingan metode:

- Metode A (`y[::faktor_potong]`): langsung mengambil setiap 6 sampel tanpa *low-pass filter*, sehingga frekuensi di atas 4.000 Hz dapat terlipat menjadi frekuensi palsu.
- Metode B (`librosa.resample`): menggunakan anti-aliasing low-pass filter sebelum downsampling, sehingga frekuensi di atas Nyquist dikurangi terlebih dahulu.

Pada spektrogram, kedua metode terlihat cukup mirip karena energi sinyal asli lebih banyak berada pada frekuensi rendah-menengah. Namun, aliasing tetap berpotensi terjadi pada Metode A.

